

BECE406E — FPGA BASED SYSTEM DESIGN

Final Project Report

Hybrid Adder-Based 12×12 Multiplication: A Combination of CSA and Carry Select Adder

Aniruddhan S — 23BLC1159

Mirunalini A — 23BLC1224

Vinay K — 23BLC1298

Submitted to

Dr. Jean Jennifer Nesam

Slot: G1

1. Introduction

Multiplication is a core arithmetic operation in Field Programmable Gate Arrays (FPGAs), especially in applications such as digital signal processing (DSP), image processing, and embedded systems. Although modern FPGAs provide dedicated DSP slices for multiplication, implementing custom multiplier architectures using hybrid adder designs is essential for understanding arithmetic optimization, resource utilization, and performance trade-offs.

In this work, a 12×12 multiplier is designed on FPGA using a hybrid adder architecture that combines a Carry Save Adder (CSA) for partial-product reduction with a Carry Select Adder (CSLA) for the final, delay-critical addition stage. Two implementations were built and compared: a bit-level design without grouping, and a grouped, hybrid design using a CSA tree with a combined Ripple Carry Adder (RCA) and Carry Select Adder final stage.

2. Multiplication on FPGA

On FPGA, multiplication is performed through three stages:

- Partial Product Generation — using AND gates.
- Partial Product Addition (lower half) — using adders such as CSA.
- Final Addition (upper half) — using faster adders such as CSLA.

Since FPGA performance is determined by propagation delay, LUT/ALM usage, and power consumption, efficient adder selection plays a key role in achieving optimized results.

2.1 Carry Save Adder (CSA)

- Implemented where multiple partial products overlap heavily.
- Maps efficiently onto LUTs because it avoids long carry chains.
- Outputs two vectors (Sum and Carry), combined later in the final addition stage.
- Advantage: low delay due to absence of carry propagation across all bits.

2.2 Carry Select Adder (CSLA)

- Implemented for the upper, more significant bits of the result.
- Precomputes results for both carry = 0 and carry = 1, selecting the correct output using multiplexers.
- Maps efficiently to parallel LUTs and multiplexers, reducing critical-path delay.
- Advantage: fast addition for higher-order bits where carry propagation would otherwise dominate.

2.3 Hybrid Adder-Based Multiplier

The proposed 12×12 multiplier combines both adder types so that CSA reduces partial products with minimal carry-chain delay, while CSLA (and a small RCA block for the least-significant final chunk) resolves the final sum quickly. This hybrid approach balances speed and logic utilization, leading to an optimized FPGA implementation.

3. Block Diagram

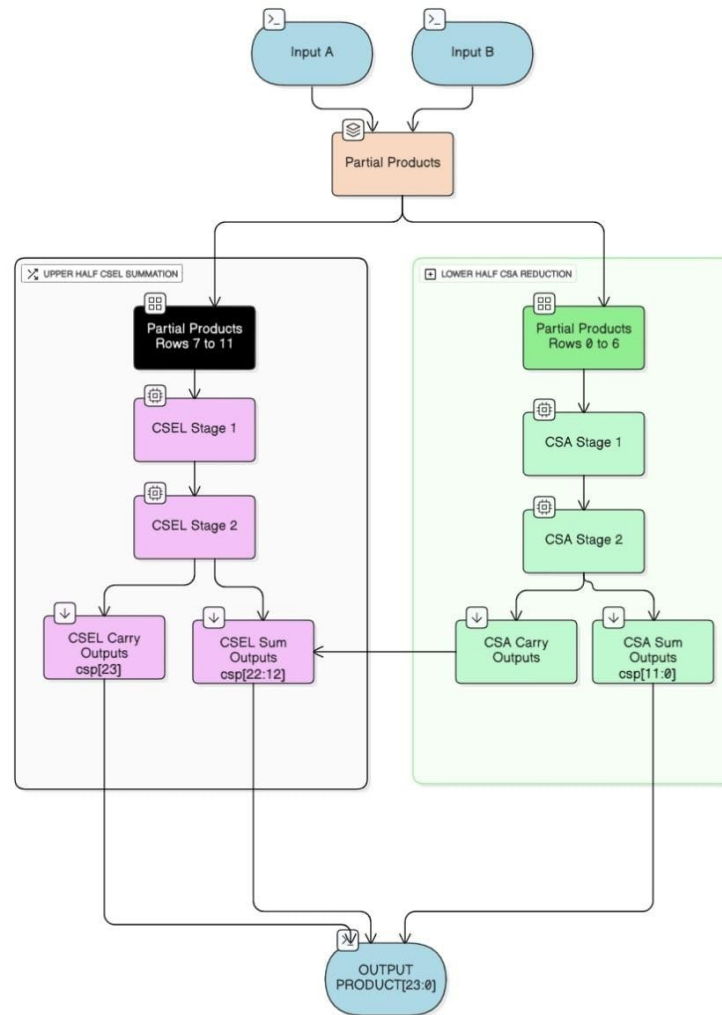


Figure 1: Block diagram of the hybrid CSA + CSLA multiplier architecture.

4. Design 1 — Without Bit Grouping

In this design, multiplication is implemented directly at the bit level without grouping or segmenting the operands. Each bit of the multiplicand is ANDed with every bit of the multiplier, forming a 12×12 matrix of partial products. These partial products are arranged diagonally according to their weight positions and summed using a structured combination of Half Adders (HA) and Carry Save Adders (CSA). The final addition is performed using a Carry Select Adder (CSLA). This approach gives clear insight into the hardware structure but consumes more LUTs/ALMs due to the larger number of bit-level operations and interconnections.

4.1 Verilog — Core Modules (excerpt)

Full source: rtl/multiplier_no_grouping.v in the GitHub repository.

```
module ha(a,b,sum,cout);
    input a,b;
    output sum,cout;
    assign sum = a^b;
    assign cout = a&b;
endmodule

module csa(a,b,cin,sum,cout);
    input a,b,cin;
    output sum,cout;
    assign sum = a^b^cin;
    assign cout = (a&b) | (a&cin) | (b&cin);
endmodule

module csel(a,b,cin,sum,cout);
    input a,b,cin;
    output sum,cout;
    wire sum0, cout0, sum1, cout1;
    assign sum0 = a^b;
    assign cout0 = a&b;
    assign sum1 = a^b^1'b1;
    assign cout1 = a|b;
    assign sum = cin ? sum1 : sum0;
    assign cout = cin ? cout1 : cout0;
endmodule

module multiplierForDa(a,b,pro);
    input [11:0] a,b;
    output [23:0] pro;
    // ... 12x12 partial-product matrix reduced via ha/csa/csel stages
    // (see full listing in rtl/multiplier_no_grouping.v)
endmodule
```

4.2 Simulation Results

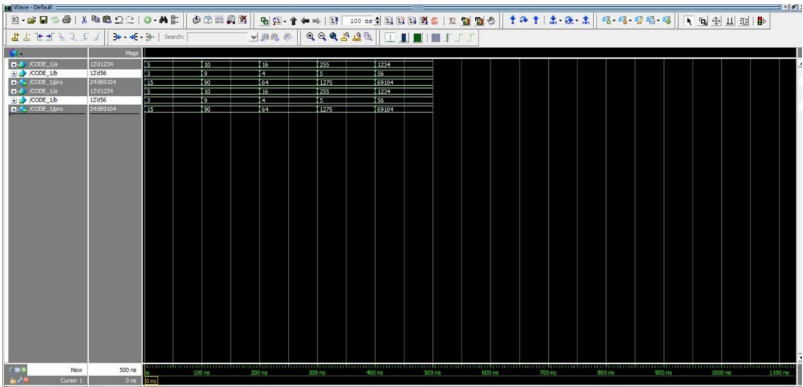


Figure 2: Simulation waveform — Design 1.

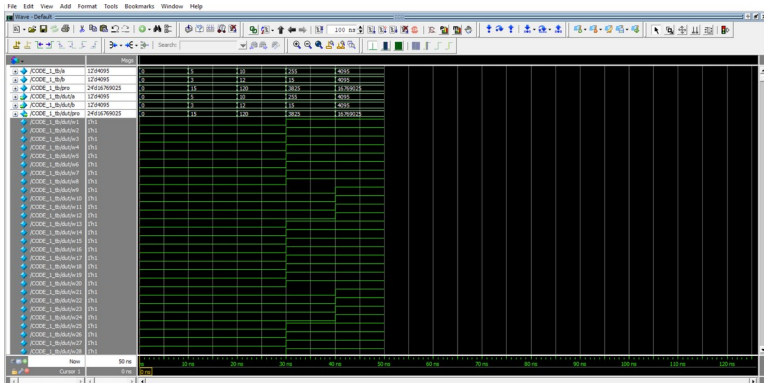


Figure 3: Simulation output — Design 1.

4.3 Synthesis Report

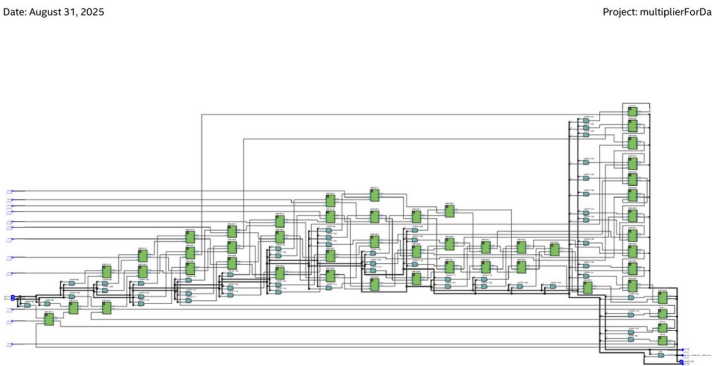


Figure 4: Synthesis summary — Design 1.

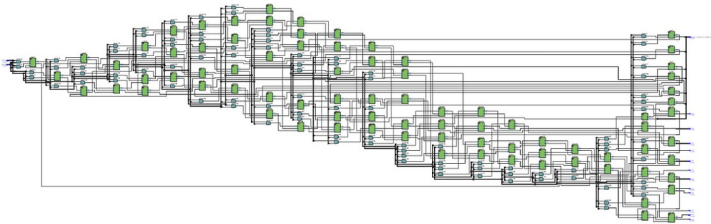


Figure 5: Resource utilization — Design 1.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Sun Aug 31 11:15:02 2025
Quartus Prime Version	24.1std.0 Build 1077 03/04/2025 SC Standard Edition
Revision Name	multiplierForDa
Top-level Entity Name	multi_12
Family	Cyclone V
Device	5CSXFC6C6U23C6
Timing Models	Final
Logic utilization (in ALMs)	221 / 41,910 (< 1 %)
Total registers	0
Total pins	48 / 342 (14 %)
Total virtual pins	0
Total block memory bits	0 / 5,662,720 (0 %)
Total DSP Blocks	0 / 112 (0 %)
Total HSSI RX PCSs	0 / 6 (0 %)
Total HSSI PMA RX Deserializers	0 / 6 (0 %)
Total HSSI TX PCSs	0 / 6 (0 %)
Total HSSI PMA TX Serializers	0 / 6 (0 %)
Total PLLs	0 / 12 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 6: Synthesis report screen — Design 1.

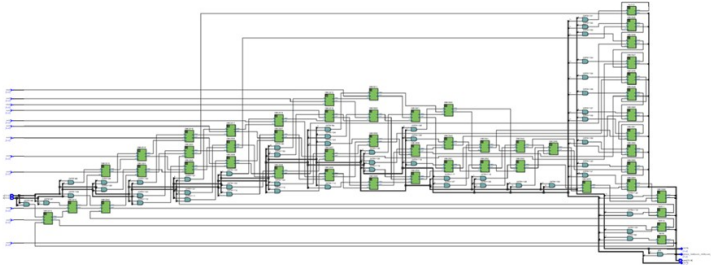


Figure 7: Synthesis report (compiler summary) — Design 1.

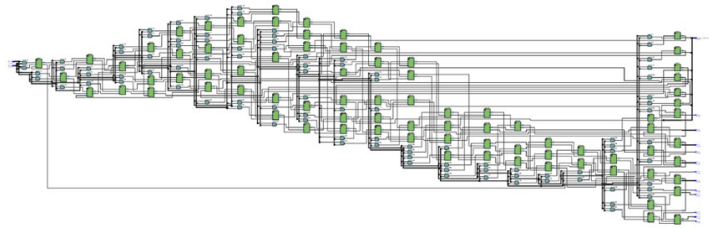


Figure 8: Fitter resource summary — Design 1.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Sun Aug 31 11:15:02 2025
Quartus Prime Version	24.1std.0 Build 1077 03/04/2025 SC Standard Edition
Revision Name	multiplierForDa
Top-level Entity Name	multi_12
Family	Cyclone V
Device	5CSXFC6C6U23C6
Timing Models	Final
Logic utilization (in ALMs)	221 / 41,910 (< 1 %)
Total registers	0
Total pins	48 / 342 (14 %)
Total virtual pins	0
Total block memory bits	0 / 5,662,720 (0 %)
Total DSP Blocks	0 / 112 (0 %)
Total HSSI RX PCSs	0 / 6 (0 %)
Total HSSI PMA RX Deserializers	0 / 6 (0 %)
Total HSSI TX PCSs	0 / 6 (0 %)
Total HSSI PMA TX Serializers	0 / 6 (0 %)
Total PLLs	0 / 12 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 9: Fitter summary detail — Design 1.

4.4 Power and Delay — Design 1

Corner Information		Summary of Paths							
	Corner	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	Slow 1100mV OC Model	0.183	dff1:ff3[q]	dff1:ff47[q]	clk	clk	14.001	-0.089	13.559
2	Slow 1100mV 85C Model	0.207	dff1:ff3[q]	dff1:ff47[q]	clk	clk	14.001	-0.095	13.529
3	Fast 1100mV 85C Model	5.946	dff1:ff25[q]	out[1]	clk	clk	14.001	-2.140	3.745
4	Fast 1100mV OC Model	6.282	dff1:ff25[q]	out[1]	clk	clk	14.001	-2.089	3.460

Power Analyzer Status	Successful - Mon Nov 10 20:56:25
Quartus Prime Version	24.1std.0 Build 1077 03/04/2025
Revision Name	12_bit_multiplier
Top-level Entity Name	multiplier_12_bit
Family	Cyclone V
Device	5CSXFC6D6F31C6
Power Models	Final
Total Thermal Power Dissipation	21352.50 mW
Core Dynamic Thermal Power Dissipation	30.83 mW
Core Static Thermal Power Dissipation	544.98 mW
I/O Thermal Power Dissipation	20776.69 mW
Power Estimation Confidence	Low: user provided insufficient tog

Figure 10: Power analyzer summary — Design 1.

5. Design 2 — With Bit Grouping (Final Hybrid Design)

In the grouped version, the 12-bit operands are processed via an 11-bit Carry Save Adder tree (csa_12) that reduces twelve partial-product rows into a sum/carry pair using cascaded stages of full adders. The final addition combines a Ripple Carry Adder (RCA) for the least-significant chunk with a Carry Select Adder for the upper, more significant bits — built recursively from 11/12/13/14/15-bit carry-select blocks composed of 3-bit and 2-bit ripple-carry adders.

This grouped structure minimizes overall critical-path delay, improves resource utilization, and offers a modular, scalable implementation. It is the final, recommended design of this project.

5.1 Verilog — Top Module (excerpt)

Full source: `rtl/multiplier_grouped.v` in the *GitHub repository*.

```
module multi_12(a,b,pro);
    input  [11:0] a,b;
    output [23:0] pro;
    // 12 partial-product rows -> 11-bit CSA tree (csa_12)
    // -> RCA for LSB chunk + 11-bit Carry-Select Adder for MSB chunk
    csa_12 adder1(w1,w2,...,w12, csa_sum, csa_carry);
    carry_select_adder_11bit adder2(w13,...,w23,w24,1'b0,
                                    Carry_select_adder_sum,
                                    carry_select_adder_carry);

    assign pro[0]      = a[0]&b[0];
    assign pro[11:1]   = csa_sum[10:0];
    assign pro[23:12]  = Carry_select_adder_sum[12:0];
endmodule
```

5.2 Simulation Results

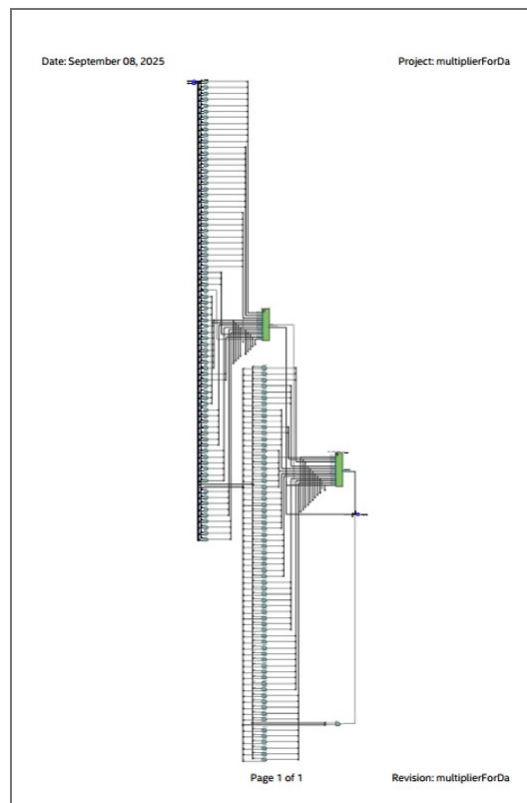


Figure 11: Simulation waveform — Design 2 (grouped).

Flow Summary	
<<Filter>>	
Flow Status	Successful - Mon Sep 8 22:05:08 2025
Quartus Prime Version	24.1std.0 Build 1077 03/04/2025 SC Standard Edition
Revision Name	multiplierForDa
Top-level Entity Name	multi_12
Family	Cyclone V
Device	5CSXFC6D6F31C6
Timing Models	Final
Logic utilization (in ALMs)	185 / 41,910 (< 1 %)
Total registers	0
Total pins	48 / 499 (10 %) 185 / 41,910 (< 1 %)
Total virtual pins	0
Total block memory bits	0 / 5,662,720 (0 %)
Total DSP Blocks	0 / 112 (0 %)
Total HSSI RX PCSs	0 / 9 (0 %)
Total HSSI PMA RX Deserializers	0 / 9 (0 %)
Total HSSI TX PCSs	0 / 9 (0 %)
Total HSSI PMA TX Serializers	0 / 9 (0 %)
Total PLLs	0 / 15 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 12: Simulation output — Design 2 (grouped).

5.3 Synthesis Report

Flow Status	Successful - Mon Nov 10 21:05:51 2025
Quartus Prime Version	24.1std.0 Build 1077 03/04/2025 SC Lite Edition
Revision Name	Multiplier_12_Bit_With_Hybrid_Adders
Top-level Entity Name	Multiplier_12_Bit_With_Hybrid_Adders
Family	Cyclone V
Device	5CSXFC6D6F31C6
Timing Models	Final
Logic utilization (in ALMs)	187 / 41,910 (< 1 %)
Total registers	0
Total pins	49 / 499 (10 %)
Total virtual pins	0
Total block memory bits	0 / 5,662,720 (0 %)
Total DSP Blocks	0 / 112 (0 %)
Total HSSI RX PCSs	0 / 9 (0 %)
Total HSSI PMA RX Deserializers	0 / 9 (0 %)
Total HSSI TX PCSs	0 / 9 (0 %)
Total HSSI PMA TX Serializers	0 / 9 (0 %)
Total PLLs	0 / 15 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 13: Synthesis summary — Design 2.

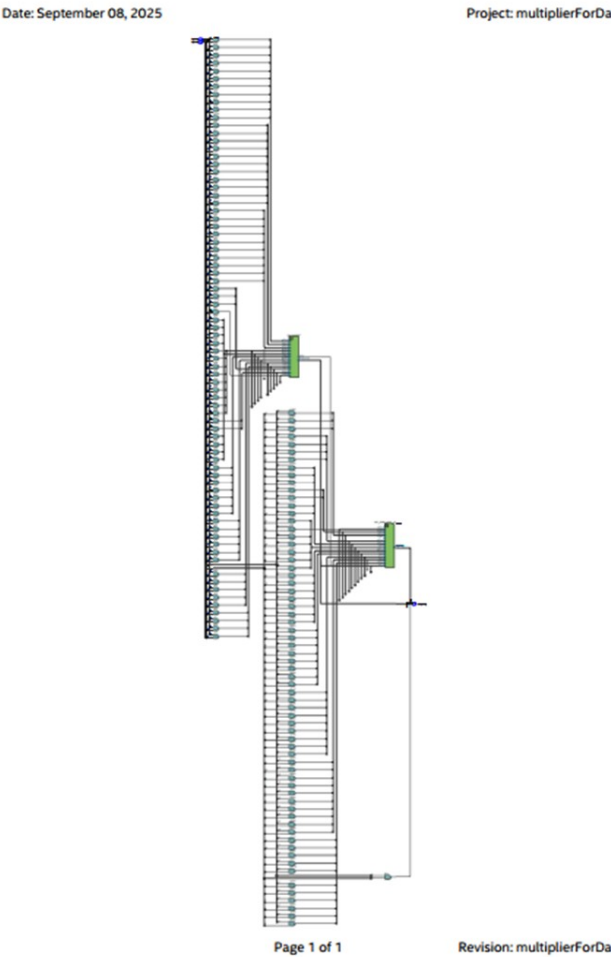


Figure 14: Resource utilization detail — Design 2.

5.4 Power and Delay — Design 2

	Corner	Slack	From Node	To Node	Launch Clock	Latch Clock	Relationship	Clock Skew	Data Delay
1	Slow 1100mV 85C Model	0.000	flipFL_ff29[q	out[5]	clk	clk	12.255	-4.042	6.043
2	Slow 1100mV 0C Model	0.197	flipFL_ff29[q	out[5]	clk	clk	12.255	-4.077	5.811
3	Fast 1100mV 85C Model	4.043	flipFL_ff29[q	out[5]	clk	clk	12.255	-2.203	3.839
4	Fast 1100mV 0C Model	4.392	flipFL_ff39[q	out[15]	clk	clk	12.255	-2.140	3.553

Power Analyzer Status	Successful - Mon Nov 10 21:05:51 2025
Quartus Prime Version	24.1std.0 Build 1077 03/04/2025 SC Lite
Revision Name	Multiplier_12_Bit_With_Hybrid_Adders
Top-level Entity Name	Multiplier_12_Bit_With_Hybrid_Adders
Family	Cyclone V
Device	5CSXFC6D6F31C6
Power Models	Final
Total Thermal Power Dissipation	21336.53 mW
Core Dynamic Thermal Power Dissipation	23.99 mW
Core Static Thermal Power Dissipation	544.98 mW
I/O Thermal Power Dissipation	20767.56 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate

Figure 15: Power analyzer summary — Design 2.

6. Results Comparison

PARAMETER	CSA + CSLA Multiplier (Code 1)	Hybrid Multiplier — Grouping (Code 2)
Logic Utilization (ALMs)	221 / 41,910	185 / 41,910
Core Dynamic Power (mW)	30.83	23.99
Core Static Power (mW)	544.98	544.98
I/O Thermal Power (mW)	20,776.69	20,767.56
Total Thermal Power Dissipation (mW)	21,352.50	21,336.53
Power Estimation Confidence	Low (insufficient toggle data)	Low (insufficient toggle data)
Timing Slack (min)	0.183 ns (Slow 0°C)	0.000 ns (Slow 0°C)
Clock Relationship	14.001 ns	12.255 ns

6.1 Area Efficiency

The hybrid CSA–Carry Select multiplier (Design 2) uses ~16% fewer ALMs (185 vs 221), showing better hardware utilization.

6.2 Power Efficiency

Design 2 consumes slightly lower total power (21.336 W vs 21.352 W) and less dynamic power (23.99 mW vs 30.83 mW), implying improved switching activity and internal optimization.

6.3 Timing

Both designs meet timing, but Design 2 operates with a shorter clock relationship (12.255 ns vs 14.001 ns), suggesting a higher potential operating frequency — at the cost of a tighter (zero) timing slack margin.

7. Conclusion

The proposed 12×12 hybrid adder-based multiplier demonstrates an efficient FPGA implementation by strategically combining Carry Save Adder (CSA) and Carry Select Adder (CSLA) architectures, with a Ripple Carry Adder handling the least-significant final chunk. By utilizing CSA where partial products overlap most, the design minimizes carry propagation during partial-product accumulation, while CSLA accelerates final addition in the most delay-sensitive region. This division of labor reduces critical-path delay, optimizes LUT/ALM utilization, and balances performance with resource consumption.

Between the two implementations, the grouped hybrid design (Design 2) is the better-performing architecture overall: fewer logic resources (185 vs 221 ALMs), lower dynamic power (23.99 mW vs 30.83 mW), and a faster clock relationship (12.255 ns vs 14.001 ns), making it the more scalable and performance-optimized choice for larger multipliers on FPGA.

8. Authors

- Aniruddhan S — 23BLC1159
- Mirunalini A — 23BLC1224
- Vinay K — 23BLC1298

BECE406E – FPGA Based System Design, VIT Chennai · Guide: Dr. Jean Jennifer Nesam